FreeSurfer-free cortical point clouds from T1w MRI

The fastest reliable pipeline for extracting a cortical surface point cloud from a T1-weighted MRI uses deepbet or HD-BET for brain extraction, ANTsPyNet's `deep_atropos` for tissue segmentation, a distance-transform-based cortical shell, Gaussian-smoothed marching cubes, and `robust_laplacian` for Laplace-Beltrami eigendecomposition. This entire chain runs in under 60 seconds on a GPU, produces surfaces within ~**0.8–1.0 mm mean distance** of FreeSurfer pial surfaces, and requires zero FreeSurfer components. Emerging deep learning tools like BrainNet (SimNIBS, 2025) now reconstruct explicit cortical meshes in ~1 second, ResearchGate but the morphological pipeline remains the most transparent and customizable approach for library integration.

GPU-accelerated brain extraction: deepbet leads on speed and accuracy

Five pip-installable brain extraction tools are production-ready. **deepbet** achieves a median Dice of **99.0** across 7,837 images from 191 OpenNeuro datasets ACM Digital Library — the highest reported — while running in roughly 1 second on GPU.

deepbet (recommended for T1w)

```python
pip install deepbet
```

```python
from deepbet import run_bet

run_bet(
    input_paths=['sub-01_T1w.nii.gz'],
    brain_paths=['sub-01_brain.nii.gz'],
    mask_paths=['sub-01_mask.nii.gz'],
    tiv_paths=['sub-01_tiv.csv'],   # total intracranial volume
    threshold=0.5,
    n_dilate=0,
    no_gpu=False
)
```

Outputs a skull-stripped brain, binary mask, and TIV estimate. (github) Works on raw T1w without preprocessing. GPU VRAM requirement is low — runs on consumer hardware in ~1–2 seconds. (Nils R. Winter) Published in *Computers in Biology and Medicine* (2024), MIT license. (GitHub)

HD-BET (best for pathological brains)

```python
pip install hd-bet  # v2.0.1, Dec 2024
```

```python
from HD_BET.run import run_hd_bet

run_hd_bet(
    input_files='sub-01_T1w.nii.gz',
    output_files='sub-01_brain.nii.gz',
    mode='accurate',      # 5 model ensemble; 'fast' uses 1
    device='0',           # GPU ID, 'cpu', or 'mps'
    tta=True,             # test-time augmentation
    save_mask=True,       # saves *_mask.nii.gz
    overwrite_existing=True
)
```

Outputs both a skull-stripped image and `*_mask.nii.gz`. Processes in **<5 seconds** on GPU (GitHub) (github) (~2–4 GB VRAM), ~2 minutes on CPU. Handles T1w, T2w, FLAIR, and T1+Gd without preprocessing. (PyPI) Outperforms FSL BET by +1.3–2.6 Dice points. Robust to tumors and resection cavities (GitHub) (github) — making it the better choice when pathology is expected. One gotcha: input must match MNI152 orientation (use `fslreorient2std` if needed).

SynthStrip (most versatile across contrasts)

```python
pip install nipreps-synthstrip  # standalone, no FreeSurfer needed
```

SynthStrip is contrast-agnostic — it works on T1w, T2w, FLAIR, CT, PET, MPRAGE, MP2RAGE, and any resolution. ScienceDirect Runs in **<1 minute on CPU** FreeSurfer with minimal VRAM on GPU. The Python interface is primarily CLI/nipype-based rather than a simple function call; for programmatic use, the nipype or pydra workflow interfaces (`pip install nipreps-synthstrip[nipype]`) GitHub are available. The `--no-csf` flag excludes CSF from the mask boundary, FreeSurfer which is useful for tighter cortical extraction.

ANTsPyNet (probability maps + tissue segmentation)

```python
pip install antspynet  # also installs antspyx
```

```python
import ants, antspynet

t1 = ants.image_read('sub-01_T1w.nii.gz')
prob_mask = antspynet.brain_extraction(t1, modality='t1')  # returns float 0
brain_mask = ants.threshold_image(prob_mask, 0.5, 1.0, 1, 0)
brain = t1 * brain_mask
```

Returns a **probability map** (not binary), which you threshold. Antsx The `modality='t1combined'` option fuses ANTs and NoBrainer models for best results. GPU-accelerated via TensorFlow; ~30–60 seconds on GPU, 1–3 minutes on CPU. Models auto-download (~100–500 MB) on first run.

Tissue segmentation separates cortical from deep gray matter

The critical upgrade from a binary brain mask to an anatomically meaningful cortical shell comes from tissue segmentation. **ANTsPyNet's** `deep_atropos` provides a 6-class segmentation that explicitly separates cortical gray matter (label 2) from deep gray matter (label 4) — exactly what is needed.

```python
result = antspynet.deep_atropos(
    t1=t1,
    do_preprocessing=True,   # N4 + denoising + brain extraction + MNI regis
    use_spatial_priors=1,    # MNI tissue priors (recommended)
    verbose=True
)

seg = result['segmentation_image']        # labels 0-6
probs = result['probability_images']      # 7 probability maps
```

Label	Tissue class
0	Background
1	CSF
2	**Cortical gray matter**
3	White matter
4	**Deep gray matter** (thalamus, caudate, putamen)
5	Brainstem
6	Cerebellum

With `do_preprocessing=True`, processing takes 5–10 minutes (dominated by N4 bias correction and affine registration). With `do_preprocessing=False` on already-preprocessed data, inference alone takes ~30–60 seconds. Antsx If you have already performed brain extraction with deepbet, you can set `do_preprocessing=False` only if the input is also bias-corrected and MNI-aligned.

SynthSeg is an alternative but has no pip package — it requires cloning from GitHub or a FreeSurfer installation. It is contrast-agnostic GitHub and runs in ~15 seconds on GPU, but its cortical GM label has reduced accuracy for "thin and convoluted structures such as the cerebral cortex" PubMed (per the original paper). For a CorticalFields library that prioritizes pip-installable dependencies, `deep_atropos` is the cleaner choice.

Five approaches to cortical shell extraction, ranked

The core challenge is isolating the ~2.5 mm thick cortical ribbon from a brain mask or segmentation. Here are the five approaches, from worst to best.

Morphological erosion (avoid for production use)

```python
from scipy.ndimage import binary_erosion
from skimage.morphology import ball

mask_eroded = binary_erosion(mask, structure=ball(1), iterations=3)
cortical_shell = mask & ~mask_eroded
```

At 1 mm isotropic resolution, 3 iterations with a spherical structuring element (`ball(1)`) removes ~3 mm of tissue. The `ball(1)` kernel produces more isotropic erosion than the default cross-shaped element. However, this approach creates **holes at sulcal fundi** where opposing walls are <6 mm apart (3 mm erosion from each side), and it produces a geometrically uniform shell that ignores the actual gray-white boundary. Useful only for quick prototyping.

Distance transform (good default without segmentation)

```python
from scipy.ndimage import distance_transform_edt, binary_fill_holes

mask_clean = binary_fill_holes(mask)
voxel_sizes = img.header.get_zooms()[:3]
dist = distance_transform_edt(mask_clean, sampling=voxel_sizes)
cortical_shell = mask_clean & (dist > 0) & (dist <= 3.0)  # 3mm shell
```

The Euclidean distance transform computes the true distance from each brain voxel to the nearest non-brain voxel. SciPy Voxels within 3 mm of the surface form the cortical shell. This handles sulci correctly — a voxel in a 2 mm-wide sulcus has distance ~1 mm to CSF, correctly placing it within the shell. No holes at sulcal fundi. The `sampling` parameter is essential for non-isotropic voxels. SciPy This is **significantly better than erosion** and only marginally more complex.

Morphological gradient (for surface-only extraction)

```python
from scipy.ndimage import morphological_gradient

gradient = morphological_gradient(mask.astype(np.float32), size=3)
boundary = gradient > 0  # 1-2 voxel thick surface
```

This extracts only the boundary — a 1–2 voxel thick shell. Too thin for a cortical ribbon, but excellent when you want the outer brain surface specifically for marching cubes (where you only need the isosurface, not a volumetric shell).

Tissue segmentation cortical GM mask (recommended)

```python
# After deep_atropos
seg_data = seg.numpy()
cortical_gm = (seg_data == 2)  # cortical gray matter only
```

This is the most anatomically accurate approach — it follows the actual tissue boundaries rather than imposing a geometric shell. Deep gray matter structures are excluded by definition. The limitation is that voxel-resolution segmentation cannot capture sub-voxel boundaries of the cortical ribbon (1–5 mm thick PubMed Central at 1 mm voxel size), but for surface extraction via marching cubes, this is sufficient.

Combined approach (best quality)

```python
# Use tissue segmentation GM probability for a smooth cortical shell
gm_prob = probs[2].numpy()  # cortical GM probability from deep_atropos
# Smooth and threshold
cortical_mask = gm_prob > 0.3  # lower threshold captures more cortex
cortical_mask = binary_fill_holes(cortical_mask)
```

Using the GM probability map rather than the hard segmentation gives a smoother boundary that produces better marching cubes output. The probability threshold can be tuned: **0.3** captures the full cortical ribbon including partial-volume voxels, while **0.5** gives a tighter, more conservative shell.

Marching cubes to point cloud: the critical parameters

Pre-smoothing is essential

Binary masks produce staircase artifacts. Apply Gaussian smoothing **before** marching cubes:

```python
from scipy.ndimage import gaussian_filter
from skimage.measure import marching_cubes

smooth = gaussian_filter(cortical_mask.astype(np.float32), sigma=0.5)
verts, faces, normals, values = marching_cubes(smooth, level=0.5, step_size=
```

Sigma = 0.5 voxels (FWHM ≈ 1.18 mm) is the sweet spot for 1 mm isotropic data. It eliminates voxel stairstepping while preserving sulcal geometry. Sigma = 1.0 begins to blur narrow sulci. Sigma > 1.5 destroys fine cortical features. The `level=0.5` threshold is standard for smoothed binary masks. The `step_size=1` parameter gives full resolution (TutorialsPoint) — at 1 mm isotropic, expect **300,000–500,000 vertices** for a whole-brain pial surface (comparable to FreeSurfer's ~300,000).

Voxel-to-RAS coordinate transformation

```python
import nibabel as nib

affine = img.affine  # 4×4 voxel-to-RAS matrix
vertices_ras = nib.affines.apply_affine(affine, verts)

# Transform normals by rotation only (no translation)
rotation = affine[:3, :3]
normals_ras = (rotation @ normals.T).T
normals_ras /= np.linalg.norm(normals_ras, axis=1, keepdims=True)
```

Always use `spacing=(1, 1, 1)` in marching cubes and apply the full NIfTI affine afterward. This is correct because `nibabel.get_fdata()` returns data in (i, j, k) array index space, and the affine maps these indices directly to RAS+ world coordinates. (Little Train's Blog) Passing `spacing=voxel_sizes` to marching cubes would scale but not rotate (TutorialsPoint) — creating a hybrid coordinate system that requires a modified affine.

Open3D post-processing

```python
import open3d as o3d

# Build mesh
mesh = o3d.geometry.TriangleMesh()
mesh.vertices = o3d.utility.Vector3dVector(vertices_ras)
mesh.triangles = o3d.utility.Vector3iVector(faces)
mesh.compute_vertex_normals()

# Poisson disk sampling for uniform point cloud
pcd = mesh.sample_points_poisson_disk(number_of_points=100000, init_factor=5

# Statistical outlier removal (adapts to local density — better for cortex)
pcd, idx = pcd.remove_statistical_outlier(nb_neighbors=20, std_ratio=2.0)

# Normal estimation
pcd.estimate_normals(
    search_param=o3d.geometry.KDTreeSearchParamHybrid(radius=5.0, max_nn=30)
)
pcd.orient_normals_consistent_tangent_plane(k=100)
```

Statistical outlier removal with `nb_neighbors=20, std_ratio=2.0` Open3D is preferred over radius-based removal because cortical point density varies (denser in sulci, sparser on gyral crowns). For normal estimation, **radius=5.0 mm with max_nn=30** captures local cortical curvature — at ~1 mm inter-vertex spacing, 30 neighbors span a ~3–5 mm patch, resolving gyral curvature (~5–15 mm radius) without over-smoothing sulci. Poisson disk sampling from the mesh is preferred over farthest-point sampling because it produces blue-noise-distributed points with approximately equal spacing. Open3D

Laplace-Beltrami eigendecomposition with robust_laplacian

```python
pip install robust_laplacian
```

```python
import robust_laplacian
import scipy.sparse.linalg as sla

# PREFERRED: use mesh_laplacian if you have faces from marching cubes
L, M = robust_laplacian.mesh_laplacian(verts_ras, faces)

# ALTERNATIVE: point cloud mode (30 neighbors default, rarely needs tuning)
L, M = robust_laplacian.point_cloud_laplacian(points, n_neighbors=30)

# Eigendecomposition: first k eigenpairs
n_eig = 50
evals, evecs = sla.eigsh(L, n_eig, M, sigma=1e-8)
```

Sharp & Crane's (2020) method constructs an intrinsic Delaunay triangulation internally, guaranteeing **nonnegative edge weights** and a positive semidefinite Laplacian — even on nonmanifold meshes and noisy point clouds. (github +2) The `mesh_laplacian` path is preferred when faces are available because it uses the actual mesh connectivity rather than constructing one from nearest neighbors. The `mollify_factor=1e-6` default handles floating-point degeneracies. (GitHub) (GitHub) The `sigma=1e-8` parameter in `eigsh` uses shift-invert mode to find the smallest eigenvalues efficiently.

FastSurfer validated that the first 3 non-constant LB eigenfunctions parameterize the cortical surface into anterior-posterior, superior-inferior, and medial-lateral axes — enabling spectral spherical embedding that replaces FreeSurfer's iterative inflation. (Deep-mi) (PubMed Central) No published neuroimaging paper has specifically used the `robust_laplacian` library on cortical surfaces, but the mathematical framework is identical to what FastSurfer and other spectral cortical analysis methods employ. (Summergeometry)

Deep learning tools that bypass this entire pipeline

Several DL methods reconstruct cortical surfaces directly from T1w in seconds, potentially replacing the full morphological pipeline.

BrainNet (SimNIBS, 2025) is the newest and most promising. (arXiv) It produces white matter and pial meshes via template deformation (arXiv) in **~1 second on GPU**, works on any MRI contrast/resolution (trained on synthetic data via domain randomization), (Springer) and achieves **~50% lower cortical thickness error** than recon-all-clinical (0.24 vs 0.50 mm). (arXiv +3) Code is public at `github.com/simnibs/brainnet`. (arXiv) (arXiv)

Vox2Cortex / V2C-Flow (CVPR 2022, MedIA 2024) predicts all 4 cortical surfaces (left/right WM and pial) from T1w in **<2 seconds**, with Hausdorff distances of ~1–2 mm versus FreeSurfer. It outputs triangle meshes with vertex correspondence to a template. Available at `github.com/ai-med/Vox2Cortex` (refactored as "vox2organ"). Requires a custom PyTorch3D fork and NiftyReg for affine registration. (GitHub)

CortexODE (IEEE TMI, 2023) uses neural ODEs for diffeomorphic surface deformation in **<5 seconds**. (PubMed) Sub-voxel accuracy validated on ADNI, HCP, and dHCP. Available at `github.com/m-qiang/CortexODE`. Not pip-installable — requires cloning and installing PyTorch3D v0.4.0. (GitHub)

CorticalFlow++ (NeurIPS 2021) reconstructs surfaces in **~1 second** via signed distance field integration. (CorticalFlow) Pre-trained models available at `github.com/lebrat/CorticalFlow`. Includes a `recon_all.sh` convenience script. (GitHub)

DeepPrep (Nature Methods, 2025) integrates FastSurferCNN + FastCSR into a full preprocessing pipeline that is **11× faster** than fMRIPrep+FreeSurfer. Validated on 55,000+ scans. (Springer Nature Experiments) Available via Docker at `github.com/pBFSLab/DeepPrep`. (GitHub) However, FastSurfer's surface module still requires a FreeSurfer license. (arXiv)

None of these are pip-installable single-function libraries yet. For a CorticalFields library that needs a clean, dependency-light pipeline, the morphological approach remains more practical. But for users who need maximum surface accuracy and can tolerate heavier dependencies, BrainNet or V2C-Flow are the strongest options.

Complete minimal pipeline

python

```python
import numpy as np
import nibabel as nib
from scipy.ndimage import distance_transform_edt, gaussian_filter, binary_fi
from skimage.measure import marching_cubes
import open3d as o3d
import robust_laplacian
import scipy.sparse.linalg as sla


def t1w_to_cortical_eigenmodes(
    t1_path: str,
    cortical_thickness_mm: float = 3.0,
    sigma: float = 0.5,
    target_points: int = 100_000,
    n_eigenmodes: int = 50,
    use_tissue_seg: bool = True,
) -> dict:
    """
    T1w.nii.gz → brain extraction → cortical shell → marching cubes
    → point cloud → normals → Laplace-Beltrami eigenmodes.

    Returns dict with keys: vertices, faces, normals, point_cloud,
    eigenvalues, eigenvectors, laplacian, mass_matrix.
    """
    # --- Step 1: Brain extraction (deepbet) ---
    from deepbet import run_bet
    import tempfile, os

    with tempfile.TemporaryDirectory() as tmp:
        brain_path = os.path.join(tmp, 'brain.nii.gz')
        mask_path = os.path.join(tmp, 'mask.nii.gz')
        run_bet([t1_path], [brain_path], [mask_path], threshold=0.5)
        mask_img = nib.load(mask_path)
        mask = mask_img.get_fdata().astype(bool)
        affine = mask_img.affine
        voxel_sizes = mask_img.header.get_zooms()[:3]

    # --- Step 2: Cortical shell ---
    if use_tissue_seg:
        import ants, antspynet
        t1_ants = ants.image_read(t1_path)
```

```
        result = antspynet.deep_atropos(t1_ants, do_preprocessing=True)
        seg = result['segmentation_image'].numpy()
        cortical_mask = (seg == 2)  # cortical GM only
        cortical_mask = binary_fill_holes(cortical_mask)
    else:
        mask_clean = binary_fill_holes(mask)
        dist = distance_transform_edt(mask_clean, sampling=voxel_sizes)
        cortical_mask = mask_clean & (dist > 0) & (dist <= cortical_thicknes

    # --- Step 3: Smooth + marching cubes ---
    smooth = gaussian_filter(cortical_mask.astype(np.float32), sigma=sigma)
    verts_vox, faces, normals_mc, _ = marching_cubes(
        smooth, level=0.5, spacing=(1., 1., 1.), step_size=1
    )

    # --- Step 4: Voxel → RAS ---
    verts_ras = nib.affines.apply_affine(affine, verts_vox)
    rot = affine[:3, :3]
    normals_ras = (rot @ normals_mc.T).T
    normals_ras /= np.linalg.norm(normals_ras, axis=1, keepdims=True)

    # --- Step 5: Open3D mesh + Poisson disk sampling ---
    mesh = o3d.geometry.TriangleMesh()
    mesh.vertices = o3d.utility.Vector3dVector(verts_ras)
    mesh.triangles = o3d.utility.Vector3iVector(faces)
    mesh.vertex_normals = o3d.utility.Vector3dVector(normals_ras)

    pcd = mesh.sample_points_poisson_disk(target_points, init_factor=5)
    pcd, _ = pcd.remove_statistical_outlier(nb_neighbors=20, std_ratio=2.0)
    pcd.estimate_normals(
        search_param=o3d.geometry.KDTreeSearchParamHybrid(radius=5.0, max_nr
    )
    pcd.orient_normals_consistent_tangent_plane(k=100)

    # --- Step 6: Laplace-Beltrami eigenmodes ---
    v = np.asarray(mesh.vertices)
    f = np.asarray(mesh.triangles)
    L, M = robust_laplacian.mesh_laplacian(v, f)
    evals, evecs = sla.eigsh(L, n_eigenmodes, M, sigma=1e-8)

    return {
        'vertices': v,
        'faces': f,
```

```
        'normals': normals_ras,
        'point_cloud': np.asarray(pcd.points),
        'point_cloud_normals': np.asarray(pcd.normals),
        'eigenvalues': evals,
        'eigenvectors': evecs,
        'laplacian': L,
        'mass_matrix': M,
    }
```

Installation for the full pipeline

```bash
pip install deepbet hd-bet antspynet nibabel scipy scikit-image open3d robus
# Ensure PyTorch is installed for your CUDA version first:
# pip install torch --index-url https://download.pytorch.org/whl/cu121
```

Validation: what accuracy to expect

Published benchmarks place mask-derived cortical surfaces at **~0.8–1.0 mm mean surface distance** and **~2.0–2.7 mm 90th-percentile Hausdorff distance** from FreeSurfer pial surfaces. These numbers come from the recon-all-clinical evaluation (Gopinath et al., 2024) and the BrainNet comparison (Nielsen et al., 2025), both using marching cubes on segmentation-derived masks versus FreeSurfer's deformable surface pipeline.

For context, FreeSurfer's own cortical thickness measurements agree with histology at **3.65 ± 0.44 mm vs 3.72 ± 0.36 mm** PubMed +2 (Magnotta et al., 2014), suggesting the ground truth itself has ~0.4 mm uncertainty. A comparative study by González-Villà et al. (2021) found that among FreeSurfer, CAT12, Laplacian, and EDT methods, the **Laplacian approach showed the least test-retest variability** (1.27%, ICC=0.89), while all methods successfully detected cortical atrophy in Alzheimer's disease. PubMed Springer

No published paper specifically validates cortical analysis via the exact brain mask → point cloud → `robust_laplacian` pipeline. The closest validation is FastSurfer's use of LB eigenfunctions for spectral spherical embedding on meshes derived from deep learning segmentation. The mathematical framework is sound — the open question is whether marching cubes mesh quality is sufficient for stable eigendecomposition, and the answer from Sharp & Crane's guarantees (nonnegative weights, PSD Laplacian) is yes. PyPI

Conclusion

The recommended pipeline for CorticalFields is: **deepbet** (1 s, DSC 99.0) (ACM Digital Library) → **deep_atropos cortical GM label** (30–60 s, separates cortical from deep GM) → **distance transform or GM probability threshold** → **Gaussian smoothing σ=0.5** → **marching cubes** (~300k–500k vertices) → **Poisson disk resampling to 100k points** → `robust_laplacian.mesh_laplacian` → **eigsh for LB eigenmodes**. Total processing time is under 2 minutes on GPU. For users wanting maximum surface accuracy without the morphological chain, BrainNet (SimNIBS, 2025) produces explicit cortical meshes in 1 second from any MRI contrast and should be monitored as a potential drop-in replacement as its ecosystem matures. (arXiv +2)